IA ORIENTADA A TAREFAS

Guia de criação de tarefas

Como escrever tasks que um agente de IA consegue planejar, em projetos já existentes e em projetos novos.

Daniel

Claude Code · MCP · Linear · ClickUp

01 · ANTES DE COMEÇAR

A task é a entrada do agente

No fluxo de IA orientada a tarefas, o agente planeja a partir do que está escrito na task. O que não estiver nela, ele precisa adivinhar ou simplesmente não considera. Por isso, escrever bem a task é a etapa que mais influencia a qualidade do plano, e é a única que depende só de você.

1 · você	1.5 · você	2 · agente	2.5 · agente	3 · você
Descreve	**Enriquece**	**Planeja**	**Registra**	**Decide**

Este guia cobre os passos **1 (Descreve)** e **1.5 (Enriquece)**: tudo o que você escreve antes de disparar o planejamento. Os passos seguintes dependem diretamente deles.

REGRA DE OURO

Descreva o problema, não a solução.

Se a task já dita a solução ("trocar a biblioteca X pela Y"), o planejamento vira só uma validação da sua ideia, e as opções alternativas deixam de existir. Quando você já tem um palpite, registre-o como **hipótese**, separado da descrição do problema. Assim o agente considera o palpite sem ficar preso a ele.

02 · ANATOMIA

As partes de uma boa task

Estas seções valem para qualquer projeto. Nem toda task precisa de todas, mas cada uma que fica de fora é uma pergunta que o plano vai responder sozinho, ou deixar em aberto.

SEÇÃO	O QUE ESCREVER	EXEMPLO
Título	Resultado observável, não a ação técnica.	Relatório exporta mais de 10 mil linhas sem travar
Problema	O que acontece hoje e o que deveria acontecer.	Hoje a exportação trava acima de 10 mil linhas; deveria gerar o arquivo em até 30 s.
Impacto	Quem é afetado, com que frequência, quanto custa.	Clientes com mais de 200 funcionários, todo fechamento de mês.
Contexto	Links ou o próprio dado: design, docs, logs, implementações anteriores.	Link da task antiga de exportação; log do erro colado.
Exceções	Casos que você já sabe que podem quebrar.	Funcionários desligados no meio do mês.
Restrições	O que não pode mudar.	O formato do arquivo é consumido por outro sistema.
Hipótese	Seu palpite, se tiver. Opcional e separado.	Talvez seja a consulta sem paginação.
Pronto quando	Critério verificável, incluindo testes.	Exporta 50 mil linhas em menos de 30 s; teste de carga cobre o caso.
Fora do escopo	O que esta task não cobre.	Novos formatos de exportação.

Link ou o próprio dado?

Prefira **link** quando a fonte muda com o tempo e o agente consegue lê-la por uma integração MCP (outra task, um doc, um arquivo de design). Prefira **colar o dado** quando ele é um retrato de um momento (log, print, payload de erro) ou quando a fonte não está integrada. Um link que o agente não consegue abrir é contexto perdido: nesse caso, cole o trecho essencial.

03 · PROJETO JÁ EXISTENTE

Aponte onde olhar e o que já se sabe

Num projeto existente, o código é a maior fonte de contexto. A task não precisa explicar o sistema: precisa dizer onde o problema vive, o que já aconteceu ali antes e o que não pode quebrar.

O que incluir além da anatomia básica

- **Onde.** Repositório, serviço, módulo, tela ou endpoint afetado. Em ambientes com vários repositórios, isso também ajuda a direcionar o planejamento para o lugar certo.
- **Histórico.** Tasks e PRs anteriores na mesma área, ADRs que limitam a solução e gotchas já conhecidos.
- **Evidência.** Passos de reprodução, logs, IDs de exemplo, versão e ambiente onde acontece.
- **O que preservar.** Comportamentos atuais que não podem regredir, contratos de API e quem consome cada um.
- **Dados em produção.** Se a mudança toca schema, índices, migração ou backfill, diga isso explicitamente. São passos irreversíveis e ficam fora da execução automática.

O que não pode faltar, por tipo de task

TIPO	ESSENCIAL
Bug	Passos de reprodução, esperado e atual, evidência e desde quando acontece.
Melhoria	Comportamento atual, comportamento desejado e por que agora.
Refatoração	Motivação concreta, limites do que não muda e como provar que nada quebrou.
Feature em vários repos	Lista de repositórios afetados, contrato entre eles e ordem de deploy.
Investigação	A pergunta a responder, o prazo e o formato da resposta. O resultado pode não ser código.

template · bug ou melhoria em projeto existente

```
## Problema
Hoje: [o que acontece]
Esperado: [o que deveria acontecer]
Reprodução: 1. [...] 2. [...] 3. [...]
Desde: [versão ou data, se souber]

## Onde
Repo / serviço: [nome]
Tela / endpoint: [nome]

## Contexto
- Task ou PR anterior: [link]
- ADR / gotcha relacionado: [link]
- Log / print: [colado abaixo ou link]

## Exceções conhecidas
- [caso]

## Preservar
- [comportamento ou contrato que não pode mudar]

## Hipótese (opcional)
- [seu palpite]

## Pronto quando
- [critério observável]
- Teste cobrindo [cenário]

## Fora do escopo
- [o que fica para outra task]
```

Mesma task, duas versões

VAGO	PLANEJÁVEL
Arrumar a exportação.	Exportação de relatório trava acima de 10 mil linhas na tela de fechamento (dashboard web).
Está lento.	A consulta leva 40 s com 50 mil linhas; o limite aceitável é 30 s. Log colado abaixo.
Trocar para streaming.	Hipótese: a consulta não é paginada. Pronto quando: 50 mil linhas em menos de 30 s, com teste de carga.

04 · PROJETO NOVO

A task carrega tudo

Sem código, não existe padrão para o agente seguir. Decisões que num projeto existente estão implícitas no código precisam estar escritas. Por isso, a primeira task de um projeto novo não é uma feature: é a fundação.

Comece pela task zero

Uma única task que descreve o projeto inteiro em alto nível. O planejamento dela não gera código: gera as primeiras decisões registradas (ADRs) e o arquivo de convenções do projeto, que todas as tasks seguintes vão usar.

- **Visão:** o que o produto faz e para quem, em um parágrafo.
- **Restrições:** prazo, plataforma, orçamento, requisitos legais ou de dados.
- **Stack:** o que já está decidido e o que está em aberto.
- **Integrações:** serviços externos, APIs e fontes de dados.
- **MVP:** o que entra na primeira versão e, principalmente, o que fica de fora.

Prepare o gerenciador antes da primeira feature

- Projeto ou workspace criado, com os status do fluxo (planejado, em progresso, em revisão, entregue).
- Prefixo de tasks definido e a ligação commit-task configurada (palavras como `Fixes ABC-12` no commit).
- Agente conectado ao gerenciador via MCP, com acesso às fontes de contexto que você vai usar.

Ordem das primeiras tasks

0 **Fundação**
Visão, restrições e MVP. Saída: ADRs iniciais e arquivo de convenções.

1 **Decisões de arquitetura**
Uma task por decisão relevante (banco, autenticação, deploy). Cada uma gera um ADR.

2 **Esqueleto**
Repositório, estrutura de pastas, CI, testes rodando e ambiente local.

3 **Primeira fatia vertical**
Uma feature pequena de ponta a ponta, que valida a arquitetura na prática.

4 **Features seguintes**
Organizadas em épicos com subtasks pequenas e dependências explícitas.

Escrevendo features num projeto novo

- **Critério de pronto mais explícito.** Sem código existente, não há padrão implícito para o agente copiar.
- **Referencie, não repita.** Aponte para os ADRs e para o arquivo de convenções em vez de reexplicar decisões em cada task.
- **Fatias pequenas.** Cada task deve ser entregável sozinha. Use épicos com subtasks e deixe a ordem de dependência escrita.
- **Planejamento simples primeiro.** Comece com o prompt inicial de planejar. Quando o mesmo processo se repetir em várias tasks, transforme-o em skill.

template · task zero (fundação)

```
## Visão
[o que o produto faz e para quem, em um parágrafo]

## Restrições
- Prazo: [...]
- Plataforma: [...]
- Requisitos legais / dados: [...]

## Stack
Decidido: [...]
Em aberto: [...]

## Integrações
- [serviço / API / fonte de dados]

## MVP
Entra: [...]
Fica de fora: [...]

## Pronto quando
- ADRs iniciais escritos para cada decisão em aberto
- Arquivo de convenções do projeto criado
- Primeiras tasks (esqueleto e primeira fatia) criadas
```

05 · LADO A LADO

Projeto existente e projeto novo

ASPECTO	PROJETO EXISTENTE	PROJETO NOVO
Fonte principal de contexto	O código	A própria task
Primeira task	O problema em si	A fundação (task zero)
Contexto essencial	Onde olhar, histórico e evidência	Visão, restrições e decisões
Lente que costuma pesar mais	Simples ou balanceada	Arquitetural
ADRs	Consultados e atualizados	Criados
Risco típico	Regressão em algo que já funcionava	Decisão implícita que ninguém registrou

06 · CHECKLIST

Antes de disparar o planejamento

- [ ] O problema está descrito sem impor a solução
- [ ] Existe um critério de pronto verificável
- [ ] O contexto está acessível: link integrado via MCP ou dado colado
- [ ] As exceções conhecidas estão listadas
- [ ] O que não pode mudar está escrito
- [ ] O fora do escopo está claro
- [ ] Passos irreversíveis (dados, schema, deploy) estão sinalizados
- [ ] O repositório ou a área afetada está indicada `EXISTENTE`
- [ ] A task referencia os ADRs e as convenções da fundação `NOVO`

07 · ERROS COMUNS

O que mais atrapalha o plano

ERRO	POR QUE ATRAPALHA	EM VEZ DISSO
Task-solução	As lentes de planejamento só validam a sua ideia.	Descreva o problema e anote o palpite como hipótese.
Task gigante	O plano fica raso e o PR fica enorme.	Quebre em subtasks entregáveis.
Contexto só no chat	Some na próxima sessão.	Traga para a task.
Link sem acesso	O agente não consegue ler.	Cole o trecho essencial.
Critério vago	"Melhorar a performance" nunca termina.	Número ou comportamento observável.
Duas coisas numa task	As decisões se misturam.	Uma task por problema.

PARA LEVAR

A IA propõe. Você decide. A task lembra. Quanto melhor ela estiver escrita, melhores são as opções que chegam até você.